

Lecture 17: Retrieval augmented generation

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will be able to...

1. Explain why language models need external knowledge
2. Describe the RAG pipeline: retrieve, augment, generate
3. Understand how embeddings enable semantic retrieval
4. Implement a basic RAG system in Python
5. Evaluate tradeoffs between RAG and fine-tuning

Announcements

Assignment 3 due this week

Wikipedia Embeddings assignment is due **Thursday, February 6 at 11:59 PM EST**.
Submit via GitHub Classroom.

Assignment 4 released

Customer Service Chatbot — build a context-aware chatbot using retrieval and generation techniques.

Due: **February 16 at 11:59 PM EST**.

The knowledge problem

Two kinds of knowledge

Parametric knowledge: Facts stored *inside* the model's parameters during training. The model "knows" things because it memorized patterns from its training data.

Non-parametric knowledge: Facts stored *outside* the model in documents, databases, or APIs. The model accesses this knowledge at inference time.

Analogy

Parametric knowledge is like what you remember from studying. Non-parametric knowledge is like having your textbook open during an exam — you can look things up when you need them.

Why parametric knowledge falls short

Limitations of relying only on model parameters

1. **Hallucination:** The model confidently generates plausible-sounding but incorrect information
2. **Stale data:** The model's knowledge is frozen at training time — it doesn't know about events after its cutoff date
3. **No citations:** The model can't tell you *where* it learned something, making it hard to verify claims
4. **Costly updates:** Retraining or fine-tuning to add new knowledge is expensive and slow
5. **Long tail:** Rare or specialized facts are poorly represented in pre-training data

What is RAG?

Retrieval Augmented Generation

RAG is a technique that gives language models access to external knowledge by *retrieving* relevant documents and *including* them in the prompt. The model generates its response based on both its parametric knowledge and the retrieved context.

Key insight

Instead of trying to store all knowledge in the model's parameters, we let the model *look things up* in a knowledge base at inference time. This separates *what the model knows how to do* (reasoning, language) from *what facts it has access to* (documents, data).

The RAG pipeline

Five steps

1. **Query:** The user asks a question
2. **Embed:** Convert the query into a vector using an embedding model
3. **Retrieve:** Find the most similar document chunks in a vector database
4. **Augment:** Insert the retrieved chunks into the prompt as context
5. **Generate:** The language model generates an answer grounded in the retrieved context

The big picture

$$\text{RAG}(q) = \text{LLM}(q + \text{retrieve}(q, \mathcal{D}))$$

where q is the query and $\mathcal{D}$ is the document collection.

Embeddings for retrieval

Connecting to lectures 11-15

Remember embeddings? In lectures 11-14, we learned that embeddings map text to vectors where **semantic similarity corresponds to geometric proximity**:

similar meaning $\Leftrightarrow$ nearby vectors

RAG uses this property for retrieval: embed the query and the documents into the same vector space, then find the documents closest to the query.

Which embedding model?

For RAG, we typically use **sentence embedding** models (not word-level) that produce a single vector for an entire passage:

- **Sentence-BERT** (all-MiniLM-L6-v2): Fast, good quality, 384 dimensions
- **OpenAI text-embedding-3-small**: API-based, 1536 dimensions
- **BGE, E5, GTE**: Open-source alternatives with strong performance

Vector similarity

Measuring how similar two texts are

Given embedding vectors a and b , we compute **cosine similarity**:

$$\text{sim}(a, b) = \frac{a \cdot b}{\|a\| \|b\|} = \frac{\sum_i a_i b_i}{\sqrt{\sum_i a_i^2} \cdot \sqrt{\sum_i b_i^2}}$$

- $\text{sim} = 1$: Identical meaning
- $\text{sim} = 0$: Unrelated
- $\text{sim} = -1$: Opposite meaning

Example

```
1 from sentence_transformers import SentenceTransformer, util
2
3 model = SentenceTransformer("all-MiniLM-L6-v2")
4
5 query = model.encode("What causes rain?")
6 doc1 = model.encode("Precipitation occurs when water vapor
condenses.")
```

Vector similarity

```
9 | print(util.cos_sim(query, doc1)) # ~0.72 (relevant!)
10| print(util.cos_sim(query, doc2)) # ~0.05 (irrelevant)
```

...continued

Why do we need chunking?

Breaking documents into pieces

Real documents are long — hundreds or thousands of pages. We can't embed an entire book as a single vector (too much information is lost). Instead, we split documents into smaller **chunks** and embed each chunk separately.

Chunking strategies

- **Fixed-size:** Split every n characters/tokens (simple but may cut mid-sentence)
- **Sentence-based:** Split on sentence boundaries (preserves meaning better)
- **Semantic:** Use topic shifts to determine chunk boundaries (best quality but most complex)
- **Recursive:** Split on paragraphs first, then sentences if chunks are still too long

Chunk size tradeoffs

Getting the chunk size right matters

Too small (e.g., individual sentences):

- Loses surrounding context
- May not contain enough information to answer the question
- Retrieves many fragments that are hard to piece together

Too large (e.g., entire chapters):

- Embedding quality degrades — too much information in one vector
- Includes irrelevant content alongside relevant content
- Uses up the language model's context window

Rules of thumb

A good starting point is **256-512 tokens** per chunk with **50-100 token overlap** between adjacent chunks. This ensures each chunk has enough context and important information at chunk boundaries isn't lost.

Vector databases

Specialized storage for embeddings

A **vector database** stores embedding vectors and provides fast **approximate nearest neighbor** (ANN) search. Instead of comparing a query to every document (slow for millions of documents), vector databases use indexing structures to find the most similar vectors in milliseconds.

Why not just use a list?

Brute-force search over n vectors takes $O(n)$ time. For 10 million chunks, that's 10 million cosine similarity computations per query. Vector databases use techniques like HNSW (Hierarchical Navigable Small World graphs) to reduce this to roughly $O(\log n)$.

Popular vector databases

Options for different use cases

Database	Type	Best for
FAISS	Library (Meta)	Research, local experiments
ChromaDB	Embedded DB	Prototyping, small-medium scale
Pinecone	Managed cloud	Production, no-ops
Weaviate	Self-hosted	Full-featured, hybrid search
Qdrant	Self-hosted	Performance-focused

For this course, **ChromaDB** or **FAISS** are the easiest to get started with — no server or API key needed.

Python: embedding documents

Step 1: embed your knowledge base

```
1 from sentence_transformers import SentenceTransformer
2
3 # Load embedding model
4 embedder = SentenceTransformer("all-MiniLM-L6-v2")
5
6 # Your knowledge base (in practice, loaded from files)
7 documents = [
8     "The transformer was introduced by Vaswani et al. in 2017.",
9     "BERT uses bidirectional attention for language understanding.",
10    "GPT models use autoregressive (left-to-right) generation.",
11    "Attention mechanisms allow models to focus on relevant context.",
```

continued...

Python: embedding documents

```
15 | # Embed all documents
16 | doc_embeddings = embedder.encode(documents, convert_to_tensor=True)
17 | print(doc_embeddings.shape) # (5, 384) – 5 docs, 384 dims each
```

...continued

Python: retrieval

Step 2: find relevant documents

```
1  from sentence_transformers import util
2
3  def retrieve(query, documents, doc_embeddings, top_k=3):
4      """Find the top_k most relevant documents for a
5      query."""
6      # Embed the query
7      query_embedding = embedder.encode(query,
8      convert_to_tensor=True)
9
10     # Compute cosine similarity against all documents
11     scores = util.cos_sim(query_embedding, doc_embeddings)
12     [0]
```

```
10
11     # Get top-k results
12     top_indices = scores.argsort(descending=True)[:top_k]
```

continued...

Python: retrieval

```
14     results = []
15     for idx in top_indices:
16         results.append({
17             "text": documents[idx],
18             "score": scores[idx].item()
19         })
20     return results
21
22     # Example query
23     results = retrieve("How do transformers work?", documents, doc_embeddings)
24     for r in results:
25         print(f" [{r['score']:.3f}] {r['text']}")
```

...continued

Python: generation with context

Step 3: augment the prompt and generate

```
1  from openai import OpenAI  # or any LLM API
2
3  client = OpenAI()
4
5  def rag_answer(query, documents, doc_embeddings):
6      """Answer a question using RAG."""
7      # Step 1: Retrieve relevant context
8      context_docs = retrieve(query, documents, doc_embeddings,
9                              top_k=3)
9      context = "\n".join([doc["text"] for doc in context_docs])
10
11     # Step 2: Build the augmented prompt
12     prompt = f"""Answer the question based on the provided
context.
```

continued...

Python: generation with context

```
13 | If the context doesn't contain the answer, say "I don't know."  
14 |  
15 | Context:  
16 | {context}  
17 |  
18 | Question: {query}  
19 | Answer: ""  
20 |  
21 | # Step 3: Generate with the LLM  
22 | response = client.chat.completions.create(  
23 |     model="gpt-4",  
24 |     messages=[{"role": "user", "content": prompt}]
```

...continued...

Python: generation with context

```
25     )  
26     return response.choices[0].message.content  
27  
28     answer = rag_answer("What is BERT?", documents, doc_embeddings)  
29     print(answer)
```

...continued

End-to-end RAG with ChromaDB

A complete mini RAG system

```
1 import chromadb
2 from sentence_transformers import SentenceTransformer
3
4 # Set up ChromaDB (persistent storage)
5 client = chromadb.Client()
6 collection = client.create_collection("course_notes")
7
8 # Add documents (ChromaDB handles embedding automatically)
9 collection.add(
10     documents=[
11         "The transformer uses self-attention to process
sequences in parallel.",
```

continued...

End-to-end RAG with ChromaDB

```
12         "Cross-entropy loss measures prediction error for classification.",
13         "RAG combines retrieval with generation for knowledge-grounded answers.",
14     ],
15     ids=["doc1", "doc2", "doc3"]
16 )
17
18 # Query – ChromaDB embeds the query and finds nearest neighbors
19 results = collection.query(query_texts=["How does attention work?"], n_results=2)
20 print(results["documents"])
21 # [['The transformer uses self-attention to process sequences in parallel.',
22 #   'RAG combines retrieval with generation for knowledge-grounded answers.']]
```

...continued

RAG vs fine-tuning

When to use which

	RAG	Fine-tuning
Knowledge updates	Easy — just update the documents	Hard — requires retraining
Cost	Cheap (no training needed)	Expensive (GPU time, data prep)
Citations	Natural — you know which docs were retrieved	Not possible
Hallucination	Reduced (grounded in retrieved text)	Can still hallucinate
Domain adaptation	Good for factual Q&A	Better for style/behavior changes
Latency	Higher (retrieval + generation)	Lower (just generation)

They're complementary

Many production systems use *both*: fine-tune the model for the domain's style and behavior, then use RAG for up-to-date factual knowledge.

Advanced RAG techniques

Beyond basic RAG

1. **Re-ranking**: After initial retrieval, use a cross-encoder model to re-score and re-order results for better precision
2. **Hybrid search**: Combine vector similarity (semantic) with keyword matching (BM25) for more robust retrieval
3. **Query expansion**: Rewrite or expand the user's query using an LLM before retrieval to improve recall
4. **Multi-hop retrieval**: For complex questions, retrieve → generate an intermediate answer → retrieve again with new context
5. **Contextual compression**: Summarize retrieved chunks to fit more information into the context window

Limitations of RAG

RAG is not a silver bullet

Retrieval quality: If the retriever fails to find relevant documents, the generator can't produce a good answer — garbage in, garbage out.

Context window limits: Even with RAG, there's a limit to how much retrieved text fits in the prompt. Long documents may need aggressive chunking or summarization.

Latency: The retrieval step adds time. For real-time applications, this overhead matters.

Reasoning over multiple sources: RAG is great for finding a specific fact, but struggles when the answer requires synthesizing information across many documents.

Embedding quality: The retriever is only as good as the embedding model. Domain-specific queries may need domain-specific embeddings.

Best practices

Making RAG work well

1. **Chunk wisely:** Experiment with chunk sizes (256-512 tokens is a good start). Use overlap between chunks.
2. **Choose the right embedding model:** Test multiple models on your specific domain. General-purpose models may not capture domain-specific semantics.
3. **Evaluate retrieval separately:** Before blaming the LLM, check if the retriever is finding the right documents.
4. **Include metadata:** Store source, date, and section info with chunks so the LLM can cite sources.
5. **Handle "I don't know":** Instruct the model to say when the context doesn't contain the answer rather than hallucinating.

Try it yourself

Interactive demo

Explore our interactive RAG demo to see retrieval augmented generation in action:

RAG System Demo — Upload documents, ask questions, and see how the system retrieves relevant passages and generates grounded answers.

Assignment 4 connection

Your next assignment asks you to build a **Customer Service Chatbot** that uses retrieval and generation techniques. The RAG concepts from today's lecture are directly applicable!

Discussion: RAG applications

Questions to consider

1. What kinds of applications benefit most from RAG? Where would fine-tuning be better?
2. A company wants to build a chatbot that answers questions about their internal documents. What are the privacy implications of using RAG vs. fine-tuning?
3. How would you evaluate whether a RAG system is working well? What metrics matter?
4. If the retrieved documents contain contradictory information, how should the system handle that?

Discussion: the future of knowledge-grounded generation

Questions to consider

1. Will RAG eventually be unnecessary if models get large enough to memorize everything?
2. How does RAG change the economics of AI — who controls the knowledge base controls the answers?
3. Could RAG be used to make language models more "honest" about what they know vs. don't know?
4. What happens when the retrieved documents themselves contain misinformation?

References

Further reading

Lewis et al. (2020, *NeurIPS*) "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" — The original RAG paper.

Borgeaud et al. (2022, *ICML*) "Improving Language Models by Retrieving from Trillions of Tokens" — RETRO: scaling retrieval to massive corpora.

Gao et al. (2024, *arXiv*) "Retrieval-Augmented Generation for Large Language Models: A Survey" — Comprehensive survey of RAG techniques.

ChromaDB Documentation — Getting started with vector databases for RAG.

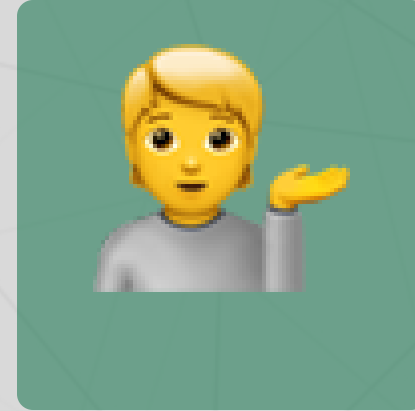
Questions?



Email



Discord



Office hours

Up next...

Week 6: BERT and encoder models — bidirectional attention and masked language modeling!